

O que realmente está exposto?

Missão: investigar uma exposição simples no Juice Shop local e produzir um registro que conecte ativo, fraqueza, vetor, consequência, controle e reteste.

Duração	Organização	Entrega
52 minutos	duplas; papéis rotativos	um PDF por dupla

Objetivos

- Inventariar os ativos do ambiente Juice Shop relacionando função e consequência.
- Distinguir ameaça, fraqueza, vulnerabilidade, exposição e exploit na evidência.
- Priorizar o registro, escolher controle e definir validação reproduzível.

Pré-requisitos e recursos

- A01 e página MkDocs A02 consultável durante a atividade.
- Docker Desktop iniciado no Windows ou Docker Engine em execução no Linux.
- Navegador com DevTools; Burp Community é opcional.
- Acesso para baixar a imagem antes da aula; depois do download, a prática pode continuar localmente.
- Este roteiro contém evidência alternativa para falha do ambiente.

ATENÇÃO

Escopo: somente o Juice Shop local fornecido. Não teste a demonstração pública, sistemas institucionais ou terceiros. Use dados fictícios. Pare se o destino divergir, houver dados reais ou o comportamento alcançar outro sistema.

Hipótese inicial

Antes de executar, registre qual recurso pode estar exposto, por que isso seria possível e qual resposta confirmaria ou refutaria sua hipótese.

Hipótese: _____

Evidência esperada: _____

Etapa 1 — Subir o Juice Shop localmente (12 min)

Os comandos Docker são iguais no PowerShell do Windows e no terminal Linux. No Windows, abra primeiro o Docker Desktop em modo de containers Linux. No Linux, confirme que o Docker Engine está ativo.

1. Confirme que o cliente consegue falar com o mecanismo Docker:

```
docker version
```

2. Baixe a imagem oficial usada pelo guia do OWASP Juice Shop:

```
docker pull bkimminich/juice-shop
```

3. Inicie um container acessível somente pela própria máquina:

```
docker run --rm -d --name juice-shop-a02 -p 127.0.0.1:3000:3000 bkimminich/juice-shop
```

4. Confirme o estado e aguarde a coluna STATUS indicar que o container está em execução:

```
docker ps --filter name=juice-shop-a02
```

ATENÇÃO

O vínculo 127.0.0.1 limita o serviço ao computador local. Não substitua por 0.0.0.0, não publique a porta no roteador e não use a demonstração pública do Juice Shop como alvo.

Windows — PowerShell

- Inicie o Docker Desktop e aguarde a indicação de que o mecanismo está ativo.
- Execute os quatro comandos acima no PowerShell, sem acrescentar sudo.
- Abra <http://127.0.0.1:3000> no navegador. Se o Docker estiver em modo de containers Windows, altere para containers Linux.

Linux — terminal

- Execute `docker version`. Se houver erro de conexão, inicie o serviço conforme a instalação da sua distribuição ou peça apoio ao professor.
- Execute os quatro comandos acima. Use `sudo` somente se essa for a configuração institucional definida para o laboratório.
- Abra <http://127.0.0.1:3000> no navegador.

PISTA

Se a porta 3000 já estiver ocupada, pare e confirme com o professor. Quando autorizado, use `-p 127.0.0.1:3001:3000` e acesse <http://127.0.0.1:3001>.

Etapa 2 — Verificar o alvo e organizar a dupla (5 min)

- Confirme que a barra de endereço mostra exatamente 127.0.0.1 e a porta escolhida.

- Registre o estado inicial sem capturar cookies, tokens ou credenciais.
- Definam papéis: investigador opera; revisor confere escopo e anota. Troquem depois da observação.
- Se a página não abrir, consulte docker logs --tail 30 juice-shop-a02. Não improvise contra outro endereço.

EVIDÊNCIA

Estado inicial: alvo sanitizado, horário relativo e confirmação de que a página inicial responde. Não capture credenciais, tokens, cookies ou IP público.

Etapa 3 — Observar sem enumerar (10 min)

Com DevTools na aba Network, observe uma requisição orientada pelo professor. Execute uma tentativa por vez. Registre método, caminho relativo, status, tipo de conteúdo e o que a resposta demonstra. Não use scanner, força bruta ou lista de caminhos.

ALTERNATIVA

Se o ambiente falhar, use: GET /ftp/acquisitions.md → HTTP/1.1 200 OK, Content-Type: text/markdown. Trate-a como evidência fictícia; não presuma conteúdo que não foi mostrado.

PISTA

Um código 200 comprova que o servidor entregou o recurso naquela requisição. Ele não prova sozinho a causa, a autoria, o alcance nem a consequência.

Etapa 4 — Inventariar e interpretar (8 min)

Preencha ao menos quatro linhas usando somente o ambiente investigado: o arquivo ou recurso observado, a aplicação/rota, os dados relacionados e o container ou plataforma. Para cada ativo, explique função, responsável provável, consequência e evidência.

Ativo/tipo	Função/proprietário	Consequência	Prioridade e evidência

Etapa 5 — Registrar a vulnerabilidade (9 min)

Descreva comportamento esperado e observado. Relacione: ativo; ameaça plausível; fraqueza provável; vulnerabilidade concreta; caminho de exposição; técnica necessária; consequência; limites da hipótese. Não invente um CVE.

Campo	Registro da dupla
Evidência reproduzível e interpretação	
Fraqueza provável e lacuna	
Vetor/ameaça dentro do escopo	
Consequência e prioridade	
Limites e próxima evidência	

DECISÃO
Escolha uma prioridade pela consequência, não pela obtenção de uma flag. Explique o que mudaria a prioridade.

Etapa 6 — Tratar e validar (8 min)

Compare as opções e recomende uma conforme a função do recurso.

Opção	Melhor uso	Evidência/entrega	Limitação/risco
Remover da área pública	recurso não deve ser servido	rota negada e publicação revisada	pode quebrar dependência
Autorizar no servidor	papel precisa acessar	casos permitido e negado	política incorreta
Reduzir conteúdo	publicação é necessária	somente dados necessários	não corrige outras rotas
Detectar acessos	correção não imediata	log/alerta testado	não impede exposição

Controle escolhido e premissa: _____

Reteste e resultado esperado: _____

Entrega

Envie no Google Classroom um PDF chamado A02-dupla-NN.pdf com: hipótese inicial; inventário priorizado; uma evidência interpretada; registro de vulnerabilidade; controle e reteste; reflexão de até cinco linhas. Uma captura sem interpretação não conta como evidência.

Critérios de sucesso

Critério	Atendeu	Parcial
Inventário	ativos, função, proprietário e consequência	lista sem priorização
Precisão	termos distintos e limites explícitos	mistura termos ou afirma causa
Evidência	reproduzível, sanitizada e interpretada	captura pouco explicada
Tratamento	controle ligado à causa e reteste	controle genérico
Conduta	escopo e parada respeitados	registro incompleto

Limpeza e encerramento

Windows PowerShell e Linux usam o mesmo comando para parar o container:

```
docker stop juice-shop-a02
```

- Confirme que `docker ps --filter name=juice-shop-a02` não lista o container em execução.
- Como foi usado `--rm`, o container é removido automaticamente após a parada; a imagem permanece para reutilização.
- Feche DevTools e descarte capturas que contenham cookies, tokens ou dados não sanitizados.
- Confirme que `http://127.0.0.1:3000` não responde mais e preserve somente o PDF sanitizado.

Extensão opcional

Proponha uma consulta de inventário que encontraria exposições semelhantes sem varrer redes ou testar terceiros. Indique fonte dos dados, responsável e critério de atualização.

Transferência

Em uma HMI industrial, qual consequência física e qual janela de manutenção mudariam a escolha entre remover, autorizar e monitorar?

Fontes oficiais

- OWASP Juice Shop — execução com Docker:
<https://pwning.owasp-juice.shop/companion-guide/latest/part1/running.html>
- Docker Desktop para Windows: <https://docs.docker.com/desktop/setup/install/windows-install/>
- Docker Engine para Linux: <https://docs.docker.com/engine/install/>
- NIST CSF 2.0 — <https://doi.org/10.6028/NIST.CSWP.29>
- CVE Program — <https://www.cve.org/>
- CWE — <https://cwe.mitre.org/>